BOXSPLOIT LABS / PRODUCT DOCUMENTATION

PANBOX

Panduan produk & operator

Control plane AppSec self-hosted. Instalasi, koneksi CI, kontrol akses, operasi, dan batas pilot yang jelas.

VERSION 0.1 · 17 OCTOBER 2026 · PILOT

© 2026 BOXSPLOIT LABS

Daftar isi

1. Gambaran umum dan arsitektur
2. Prasyarat dan sistem operasi
3. Instalasi lokal di Debian/Linux/macOS
4. Instalasi lokal di Windows
5. Repositori SCM, agent native, dan CLI
6. SCM publik menuju PANBOX privat
7. RBAC dan akses repositori
8. API key, access key, dan scan key
9. Dashboard, findings, gate, dan laporan
10. Operasi, keamanan, dan ukuran resource
11. Troubleshooting
12. Batas pilot dan langkah berikutnya

Panduan ini menjelaskan perilaku pilot yang sudah ada. Rencana pengembangan diberi label terpisah.

© 2026 BOXSPLOIT LABS

Versi panduan: 0.1 (pilot, 17 Oktober 2026). PANBOX adalah control plane AppSec self-hosted dari BOXSPLOIT LABS. Panduan ini menjelaskan kemampuan yang sudah ada, cara menjalankannya, dan batas pilot secara eksplisit. Halaman web dan PDF adalah dokumentasi produk, bukan laporan kerentanan.

1. Gambaran umum dan arsitektur

PANBOX menerima hasil pemindaian native dari agent panbox, menyimpan snapshot per repositori dan branch, mengevaluasi quality gate, lalu menampilkan posture, findings, riwayat scan, ekspor teknis, RBAC, dan audit. Server Go menyajikan API dan UI React; PostgreSQL menyimpan data. Agent berjalan di mesin pengembang atau runner CI, bukan di container dashboard.

Gambaran infrastruktur dan alur data:

```
Host Git + CI --checkout--> runner terlindungi + agent PANBOX
                                |-- VACCIN: aturan source
                                |-- ENETRON: paket tetap --HTTPS--> data OSV
                                +-- bundle temuan + scan key --> API Go PANBOX
                                                                   |--> PostgreSQL
Browser operator --loopback/HTTPS--> UI React -----------------+
```

Host PANBOX privat menyajikan UI/API dan menyimpan data di PostgreSQL; server tidak memindai source. Runner mengirim nama dan versi paket ke API advisory, sedangkan bundle hasil dikirim ke PANBOX. Runner lokal, SSH relay, atau impor artifact belakangan dapat menghubungkan CI publik ke host privat; hanya dua jalur pertama menghasilkan gate CI sinkron. Lihat docs/architecture.md untuk batas kepercayaan dan setiap jalur jaringan.

VACCIN adalah engine aturan source code buatan PANBOX untuk beberapa pola Python, JavaScript/TypeScript, dan Go. ENETRON adalah parser dependensi versi tetap dan pencocok advisory buatan PANBOX untuk npm lockfile v2/v3, requirements.txt, dan go.mod. ENETRON mengambil data kerentanan dari API HTTPS yang kompatibel dengan OSV; OSV adalah sumber advisory, bukan executable scanner. Agent tidak menjalankan Semgrep, Grype, atau scanner lain. Cakupan ini sengaja sempit dan belum mencakup analisis dataflow luas.

Alur: source Git -> agent PANBOX -> aturan native dan pencocokan dependensi -> bundle normalisasi -> API PANBOX -> PostgreSQL -> dashboard dan keputusan gate. Modul yang gagal atau tidak lengkap tetap partial/unverified dan tidak menghapus snapshot sukses terakhir. Jika tidak ada manifest yang didukung, ENETRON berstatus error, bukan diam-diam pass.

Nilai khas saat ini adalah alur single-tenant ringan dengan logika deteksi milik PANBOX, riwayat repo/branch, status unverified eksplisit, RBAC/key terbatas, serta jalur dari SCM publik ke dashboard privat. Ini bukan klaim cakupan deteksi yang lebih luas. RIDER (secret), TESLA (IaC), AI, MCP, recasting severity, dan template laporan lanjutan belum aktif pada pilot ini.

2. Prasyarat dan sistem operasi

Dashboard: Docker Engine/Desktop dengan Linux containers dan Docker Compose v2. Ini dapat dijalankan di Debian/Linux, macOS, atau Windows; target produksi awal adalah Debian. Host tidak perlu menginstal Node atau Go untuk jalur Docker. Agent dipasang pada runner/developer host, bukan server dashboard.

Port awal: HTTP 8080 dan PostgreSQL 5434, keduanya diikat ke loopback host oleh Compose. Ubah PANBOX_HOST_PORT atau PANBOX_DB_HOST_PORT di .env bila perlu. Siapkan penyimpanan persisten untuk volume PostgreSQL. PANBOX_BOOTSTRAP_PASSWORD hanya dipakai saat database pertama kali dibuat.

3. Instalasi lokal di Debian/Linux/macOS

Dari root folder PANBOX:

```
sh scripts/init-local.sh
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
```

© 2026 BOXSPLOIT LABS

```
docker compose --env-file .env -f deploy/docker-compose.yml ps
curl -fsS http://127.0.0.1:8080/readyz
```

init-local.sh memakai /dev/urandom, membuat secret acak, membatasi izin file melalui umask 077, dan menolak overwrite .env. Untuk port lain, set PANBOX_HOST_PORT=8092 sebelum menjalankan script. Buka http://127.0.0.1:8080 atau port pilihan, lalu masuk memakai PANBOX_BOOTSTRAP_EMAIL dan PANBOX_BOOTSTRAP_PASSWORD di .env. Jangan mengirim atau commit file itu.

Untuk fixture demo opsional: PANBOX_SERVER=http://127.0.0.1:8080 bash scripts/seed-demo.sh. Script memerlukan curl dan jq, meminta password admin, serta memuat dua repo dan empat snapshot fiktif. Fixture bukan bukti kerentanan nyata.

4. Instalasi lokal di Windows

Gunakan Docker Desktop pada mode Linux containers. Di PowerShell dari root PANBOX:

```
./scripts/init-local.ps1
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
docker compose --env-file .env -f deploy/docker-compose.yml ps
Invoke-RestMethod http://127.0.0.1:8080/readyz
```

Gunakan ./scripts/seed-demo.ps1 bila membutuhkan data demo. Script Windows dan Unix menulis format .env yang sama; jalankan hanya salah satunya pada instalasi baru.

5. Repositori SCM, agent native, dan CLI

Di Repositories, pilih GitHub, GitLab, Bitbucket, Azure Repos, Gitea, atau Other Git, lalu masukkan ID/path repositori stabil dari provider tersebut. PANBOX memberi internal repository ID; gunakan untuk --repo dan variabel CI PANBOX_REPOSITORY_ID. Buat scan key khusus repo dan simpan sebagai PANBOX_TOKEN di secret manager CI. Teks Git remote bukan identitas otorisasi.

Unduh arsip agent untuk Linux, macOS, atau Windows (amd64/arm64) dari https://github.com/alterwriter/PANBOX/releases, verifikasi dengan SHA256SUMS, lalu letakkan panbox/panbox.exe pada PATH runner. Agent berupa satu binary Go dengan aturan bawaan. ENETRON mengirim nama dan versi paket tetap ke API OSV secara default; gunakan PANBOX_ADVISORY_URL atau --advisory-url untuk mirror kompatibel OSV yang disetujui. Tidak perlu binary scanner eksternal.

```
panbox ready --server http://127.0.0.1:8080
panbox go --server http://127.0.0.1:8080 --repo "$PANBOX_REPOSITORY_ID" --path .
panbox shutdown
```

ready opsional. go mendukung --branch, --commit, --base-sha, --modules vaccin,enetron, --format text|json|sarif, dan --offline --output panbox-results.json. Offline berarti tanpa upload ke dashboard; ENETRON tetap menghubungi API advisory kecuali hanya VACCIN yang dipilih. Exit code: 0 pass, 1 gate fail, 2 scan/transport/konfigurasi tidak lengkap, 3 autentikasi atau izin ditolak. shutdown hanya membersihkan cache sementara CLI.

6. SCM publik menuju PANBOX privat

Repo publik GitHub, GitLab, Bitbucket, Azure Repos, Gitea, atau Git lain dapat dipindai sementara PANBOX tetap di jaringan privat tanpa IP publik atau domain. Ada tiga pola koneksi:

- Runner lokal terlindungi: runner polling SCM melalui HTTPS keluar dan mengirim hasil ke PANBOX di jaringan lokal. Quality gate sinkron. Contoh GitHub Actions dan GitLab ada di deploy/github-actions-local-runner.yml dan deploy/gitlab-ci-local-runner.yml.
- SSH relay: host PANBOX membuka reverse tunnel ke relay SSH yang dapat dijangkau; shared runner membuka forward tunnel ke relay. PANBOX tidak menerima traffic internet masuk. Gate sinkron;

© 2026 BOXSPLOIT LABS

deploy/gitlab-ci-ssh-relay.yml adalah contoh GitLab untuk pola yang netral provider ini.

- Pull artifact: CI mengunggah bundle, lalu host PANBOX mengunduh dan mengimpor dengan panbox go --bundle panbox-results.json. Jalur ini asinkron dan tidak dapat memblokir CI. panbox relay-pull mengotomasi kasus GitLab Jobs API; untuk SCM lain gunakan API artifact atau mekanisme sinkronisasi SCM tersebut sebelum impor bundle yang sama.

Lindungi runner dan branch, dan jangan berikan scan key kepada job pull request yang tidak tepercaya. Pin SSH host key dan batasi identity relay. Repo publik belum tentu membuat artifact publik; gunakan token baca terbatas bila perlu.

7. RBAC dan akses repositori

Role admin adalah yang tertinggi dan tidak dapat diedit. Admin dapat membuat role custom dengan izin eksplisit, menambahkan user, mengganti role, dan memberi atau mencabut membership per repo di Settings & Audit. Role bawaan meliputi security_reviewer, developer, executive_viewer, dan scanner_ci. Akses data memerlukan izin operasi serta akses repo terkait. Role custom tidak boleh memperoleh rbac:manage.

Izin pilot: repos:read, repos:write, scans:read, scans:write, findings:read, reports:export, rbac:manage, tokens:manage, dan audit:read. Uji user dengan hak paling kecil sebelum memakai integrasi produksi. Perubahan role atau membership membatasi token yang sudah ada pada permintaan berikutnya.

8. API key, access key, dan scan key

Di Settings & Audit -> API & access keys, pilih label, tipe, expiry, scope, dan repositori bila perlu. API key dapat diberi scope integrasi yang dipilih. Access key hanya untuk operasi baca. Scan key dibatasi ke scans:write dan satu repositori. Nilai secret hanya terlihat sekali; simpan di secret manager, bukan source atau .panbox.yml. Key disimpan sebagai hash, bisa dicabut, dan punya masa berlaku.

Contoh tes baca API:

```
curl -H "Authorization: Bearer $PANBOX_ACCESS_KEY" http://127.0.0.1:8080/v1/repos
```

Browser memakai login password, session cookie HttpOnly, dan CSRF token. api/openapi.yaml adalah kontrak endpoint pilot. Rotasi key dilakukan dengan membuat key baru, mengganti secret di klien/CI, memastikan koneksi berhasil, lalu mencabut key lama.

9. Dashboard, findings, gate, dan laporan

Overview menampilkan cakupan repo, hitungan severity, status gate, dan waktu scan. Repositories untuk enrollment/key; Findings untuk filter repo/branch/modul/severity; Scans untuk riwayat dan rincian gate; Reports untuk ekspor teknis CSV/JSON; Settings & Audit untuk user, RBAC, key, dan log. Tema light/dark tersedia.

Severity standar tetap critical, high, medium, low, info. ARK adalah label tampilan untuk Critical/High dan BUG-STER untuk Medium/Low. Raw dan effective severity disimpan, tetapi recasting interaktif belum ada. Partial scan tidak dianggap clean; dashboard mempertahankan last successful scan. Ekspor teknis dibatasi oleh izin reports:export.

10. Operasi, keamanan, dan ukuran resource

Compose mengikat server dan PostgreSQL ke 127.0.0.1. Untuk akses dari luar host, gunakan reverse proxy HTTPS dengan kontrol akses atau salah satu jalur privat di atas; set PANBOX_SECURE_COOKIE=true bila browser disajikan lewat HTTPS. Simpan backup volume/database dan uji restore sebelum pilot penting. Jangan membuka PostgreSQL ke internet.

© 2026 BOXSPLOIT LABS

Target pilot adalah dashboard plus PostgreSQL pada host 2 GB, dengan satu ingest scan pada satu waktu dan tanpa model AI lokal. Pengukuran fixture kecil di Docker Desktop Windows: server sekitar 5,1 MiB, PostgreSQL 37,3 MiB, median API 7,1 ms dan p95 9,3 ms untuk 100 request berurutan. Angka ini bukan jaminan Debian 2 GB; agent pada runner memerlukan sizing sendiri. Ukur docker stats --no-stream dan latency pada host target dengan data representatif.

11. Troubleshooting

- /readyz gagal: periksa docker compose --env-file .env -f deploy/docker-compose.yml ps dan logs db server; pastikan PostgreSQL sehat.
- Login gagal setelah mengganti .env: bootstrap password hanya dibuat saat database pertama kali diinisialisasi. Jangan hapus volume untuk reset rutin.
- Gate exit 2: periksa manifest yang didukung, API advisory, path kerja, endpoint, dan log runner. Partial scan tidak sama dengan pass.
- Gate exit 3: periksa expiry/revocation scan key, scope scans:write, internal repo ID, role, dan membership.
- Runner Docker tidak dapat mencapai 127.0.0.1: loopback di container mengarah ke container itu sendiri; gunakan alamat privat host/container network yang dapat dijangkau.
- Pull artifact tertinggal: periksa timer, GITLAB_READ_TOKEN, ref yang diizinkan, akses Jobs API/artifact, dan state file relay.

12. Batas pilot dan langkah berikutnya

Yang tersedia: server/UI, snapshot repo/branch, pemeriksaan source bawaan VACCIN, pencocokan dependensi bawaan ENETRON dengan data OSV, gate, RBAC, key, audit, laporan teknis CSV/JSON, contoh koneksi publik-ke-privat untuk GitHub dan GitLab, serta impor bundle generik. Validasi runner SCM nyata, ketersediaan OSV atau mirror, beban Debian, backup/restore, TLS, dan rotasi credential pada lingkungan target.

Cakupan deteksi masih sempit: beberapa pola source dan manifest npm/PyPI/Go dengan versi tetap. ENETRON memerlukan jaringan ke API advisory dan menandai severity info bila data severity tidak tersedia. RIDER, TESLA, recasting severity dan approval, template PDF findings/eksekutif, AI Security, MCP, OIDC, dan rate limiting API belum tersedia. PDF ini adalah panduan produk, bukan fitur report PANBOX.

© 2026 BOXSPLOIT LABS